



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

Kafka Rebalancing

The senior engineer answer sheet

Every consumer healthy. The dashboard green. The lag climbing anyway.

Nothing crashed, so nothing points at a cause. That is what makes a rebalance the Kafka failure that looks random.

This sheet is the written form of the episode: what a rebalance actually moves, what triggers one, why a healthy consumer gets evicted for being slow, and where the duplicate comes from.

Every technical statement below was checked against Apache Kafka 4.3 documentation, named beside it. Where something is a general principle or an operational habit rather than documented behaviour, it says so.

In scope

- What a rebalance redistributes, and what it does not
- The four things that trigger one
- Heartbeats against the poll interval
- Where the duplicate comes from
- Eager, cooperative, and the Kafka 4 protocol
- Static membership, and when it helps
- What to monitor, and what will mislead you

Out of scope

- Broker internals and the group coordinator implementation
- Exactly-once semantics and transactions
- Kafka Streams rebalancing, which has its own rules
- Cluster sizing and partition count planning

Saying what you are not covering is itself a senior move. A candidate who answers everything answers nothing precisely.

01 Worked assumptions

Topic partitions	12
Consumers in the group	4
Partitions per consumer	3
Deploys per day	6, so 6 rebalances minimum
max.poll.records default	500
Slow record processing	40 ms each
Time inside one poll	$500 \times 40 \text{ ms} = 20 \text{ seconds}$

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 What a rebalance actually moves

Not the data. Not the topic. Not the offsets. A rebalance moves partition ASSIGNMENTS between the members of one consumer group.

Partition 0 was being read by consumer A. After the rebalance it is being read by consumer B. The messages did not move and the commits did not move. Only the question of who is responsible.

Almost every confusing symptom follows from that one sentence.

A consumer group is a set of consumers that cooperate to consume a topic, and partitions are divided among the members so that each partition is consumed by exactly one member of the group.

Kafka 4.3, Design: Message Delivery Semantics

Offsets are committed per partition, not per consumer, which is why a reassignment resumes from the last committed position rather than from wherever the previous owner had reached in memory.

Kafka 4.3, Design: Message Delivery Semantics

04 What triggers one, and only one of these is a failure

A member joins. You scaled up, or a pod restarted.

A member leaves cleanly, by calling close.

A member is declared dead, having stopped heartbeating within the session timeout.

A member is declared too slow, having not called poll again within the max poll interval.

Deploys trigger rebalances. Autoscaling triggers rebalances. These are normal and expected, which is exactly why the lag spike that comes with them gets dismissed as noise.

session.timeout.ms is the timeout used to detect client failures when using the group management facility; if no heartbeats are received by the broker before it expires, the broker removes the client from the group and initiates a rebalance.

Kafka 4.3, Consumer Configs

max.poll.interval.ms is the maximum delay between invocations of poll when using consumer group management; if poll is not called before it expires, the consumer is considered failed and the group rebalances.

Kafka 4.3, Consumer Configs



05 The distinction that explains most incidents

`max.poll.interval.ms` is not a heartbeat.

The heartbeat runs on a background thread. Your consumer can be heartbeating perfectly, on time, every time, while the thread that processes messages is stuck inside one slow batch.

The coordinator sees the heartbeats and knows the process is alive. It also sees that poll has not been called in far too long, and evicts the member anyway.

So the consumer is not thrown out for being dead. It is thrown out for being SLOW. And the log line says the member left the group, which reads exactly like a crash and is not one.

This is why the fix is so often to do less work per poll rather than to raise the timeout. Lower `max.poll.records`, move the slow call off the poll thread, or make the processing faster. Raising the timeout hides the symptom and lengthens the outage on the day the consumer really does die.

Heartbeats are sent by a background thread, separate from the thread that calls poll, so liveness of the process and progress of the application are detected independently.

Kafka 4.3, Consumer Configs

Raising `max.poll.interval.ms` to accommodate slow processing also increases the time before a genuinely failed consumer is detected and its partitions reassigned.

First principles, not vendor documentation

06 Where the duplicate comes from

A consumer reads a batch. It processes the batch. Before it commits, it is evicted, for either of the reasons above. The partition is reassigned. The new owner starts from the last committed offset, which is BEFORE that batch.

It processes the batch again.

This is not a bug in Kafka and it is not a race you can tune away. It is the direct consequence of at-least-once delivery plus a reassignment between processing and committing.

The only real defence is that the work is idempotent, which is the same conclusion the payments episode reaches by a different road.

Kafka guarantees at-least-once delivery by default: a consumer that processes a message and fails before committing its offset will see that message again after the offset is next read.

Kafka 4.3, Design: Message Delivery Semantics

Making the consumer side effect idempotent, keyed on something stable in the message, removes the consequence of reprocessing without removing the reprocessing.

First principles, not vendor documentation

07 Eager, cooperative, and what changed in Kafka 4.0

Under the classic eager protocol a rebalance is a global synchronisation barrier. Every member revokes everything it owns, everyone rejoins, the leader computes a new assignment, and only then does anyone resume. Stop the world, redistribute, start again.

With a handful of consumers this is a blip. With a large group and a slow assignor it is an outage, and it happens every time you deploy.

Cooperative sticky assignment softened this by revoking only the partitions that actually change hands. It helped, and it was still built on the same join and sync round trip.

Kafka 4.0 made the new consumer rebalance protocol generally available, and it removes the barrier rather than working around it. Coordination also moves: heartbeating, session timeouts and assignment become the broker's job.

ONE DETAIL WORTH GETTING RIGHT, because anyone running a 4.x broker will correct you otherwise. Server side, the new protocol is enabled by default. It is the CLIENT that must opt in.

The new consumer rebalance protocol "decreases rebalance times, thanks to its fully incremental design, which no longer relies on a global synchronization barrier."

Kafka 4.3, Consumer Rebalance Protocol

A consumer selects the new protocol with `group.protocol=consumer`; the Java consumer uses the classic protocol unless it is set.

Kafka 4.3, Consumer Configs

Under the new protocol `heartbeat.interval.ms` and `session.timeout.ms` are no longer used by the consumer, because the broker owns those decisions.

Kafka 4.3, Consumer Configs

The protocol moves group membership and assignment computation to the group coordinator on the broker, rather than electing a consumer as leader to compute the assignment.

KIP-848, The Next Generation of the Consumer Rebalance Protocol



08 Static membership, and when it actually helps

Give a consumer a stable `group.instance.id` and the group stops treating a restart as a departure. Within the session timeout the same instance comes back and picks up the same partitions, with no reassignment at all.

This is the right tool for ROLLING DEPLOYS of a fixed set of pods, where membership is genuinely the same before and after.

It is the wrong tool for AUTOSCALING, where membership really is changing and pretending otherwise just delays the inevitable rebalance.

It also has a real cost: if an instance dies for good, its partitions sit unowned until the session timeout expires. You are trading a faster deploy for a slower recovery.

`group.instance.id` is a unique identifier of the consumer instance provided by the end user; a consumer configured with one is treated as a static member.

Kafka 4.3, Consumer Configs

A static member that restarts and rejoins within the session timeout keeps its previous assignment without triggering a rebalance.

Kafka 4.3, Static Membership

A static member that does not return before the session timeout expires has its partitions reassigned, so the timeout is the upper bound on recovery time.

Kafka 4.3, Static Membership

09 What to monitor, and what will mislead you

Consumer lag alone will not tell you this story. Lag rises during a rebalance and falls afterwards, which looks like ordinary traffic variation.

Watch the rebalance RATE: how often, and does it spike with deploys. Watch rebalance LATENCY: how long the group is not working. Watch TIME BETWEEN POLLS against your max poll interval. Watch COMMIT FAILURES, which are the signature of committing after eviction.

A rebalance rate that is flat and low is a healthy group. One that climbs with traffic means you are being evicted for slowness, and no amount of scaling up will fix it, because every new member triggers another rebalance.

The consumer exposes rebalance rate and rebalance latency metrics through its own metrics interface, alongside records-lag.

Kafka 4.3, Consumer Configs

Alerting on lag alone hides rebalance storms, because the lag signature of a rebalance is indistinguishable from a traffic spike at the resolution most dashboards use.

Operational practice, not vendor documentation

10 The interview answer

A rebalance redistributes partitions among the members of a consumer group. It is triggered by membership change, by a missed heartbeat, or by exceeding the max poll interval.

The last of those is the one that bites, because a consumer can be perfectly alive and still be evicted for taking too long inside poll, and the resulting reassignment reprocesses anything handled but not yet committed.

Kafka 4.0's new protocol removes the global synchronisation barrier that made this a stop the world event, though the client has to opt in to it.

Then say what you would DO: make the processing idempotent, keep the poll loop short, and watch the rebalance rate rather than the lag.

Naming the trigger, the consequence and the mitigation in that order is what distinguishes an answer from a definition. The definition alone is a junior answer.

First principles, not vendor documentation



Questions to ask before you answer

1. Which rebalance protocol is this group on?
2. What is max.poll.interval.ms, and how long does one poll actually take?
3. How many records come back per poll?
4. Is the processing idempotent, and keyed on what?
5. When are offsets committed, before or after the side effect?
6. How many rebalances happen on an ordinary day?
7. Is this a fixed set of pods, or does it autoscale?
8. What happens to an in flight batch when a partition is revoked?

Everything else

Site

<https://code-with-sam-dev.github.io>

This episode

<https://youtu.be/VKOB2v1K0>

Written version

<https://code-with-sam-dev.github.io/blog/kafka-rebalancing-explained/>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, the Apache Software Foundation.

Technical claims verified 2026-09-12 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.